9.2 Fault tolerance with Spring Cloud Circuit Breaker and Resilience4J

As you know, resilience is a critical property of cloud native applications. One of the principles for achieving resilience is blocking a failure from cascading and affecting other components. Consider a distributed system where application X depends on application Y. If application Y fails, will application X fail, too? A circuit breaker can block a failure in one component from propagating to the others depending on it, protecting the rest of the system. That is accomplished by temporarily stopping communication with the faulty component until it recovers. This pattern comes from electrical systems, for which the circuit is physically opened to break the electrical connection and avoid destroying the entire house when a part of the system fails due to current overload.

In the world of distributed systems, you can establish circuit breakers at the integration points between components. Think about Edge Service and Catalog Service. In a typical scenario, the circuit is *closed*, meaning that the two services can interact over the network. For each server error response returned by Catalog Service, the circuit breaker in Edge Service would register the failure. When the number of failures exceeds a certain threshold, the circuit breaker trips, and the circuit transitions to *open*.

While the circuit is open, communications between Edge Service and Catalog Service are not allowed. Any request that should be forwarded to Catalog Service will fail right away. In this state, either an error is returned to the client, or fallback logic is executed. After an appropriate amount of time to permit the system to recover, the circuit breaker transitions to a *half-open* state, allowing the next call to Catalog Service to go through. That is an exploratory phase to check if there are still issues in contacting the